# EXPERIMENT 6

AIM-Create a stored procedure transfer_employee(emp_id,new_dept_id) with validation and error handling. Implement triggers for salary validation and audit logging. Test edge cases.

**Step 1: Create Stored Procedure**

```
DROP PROCEDURE IF EXISTS transfer_employee;

DELIMITER $$

CREATE PROCEDURE transfer_employee(
    IN p_emp_id INT,
    IN p_new_dept_id INT
)
BEGIN
    DECLARE v_emp_count INT DEFAULT 0;
    DECLARE v_dept_count INT DEFAULT 0;
    DECLARE v_current_dept INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    -- Check employee
    SELECT COUNT(*)
    INTO v_emp_count
    FROM Employee
    WHERE EmpID = p_emp_id;

    IF v_emp_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee does not exist';
    END IF;

    SELECT COUNT(*)
    INTO v_dept_count
    FROM Department
    WHERE DeptID = p_new_dept_id;

    IF v_dept_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Department does not exist';
    END IF;

    -- Get current department
    SELECT DeptID
    INTO v_current_dept
    FROM Employee
    WHERE EmpID = p_emp_id;

    -- Check same department
    IF v_current_dept = p_new_dept_id THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee is already in this department';
    END IF;

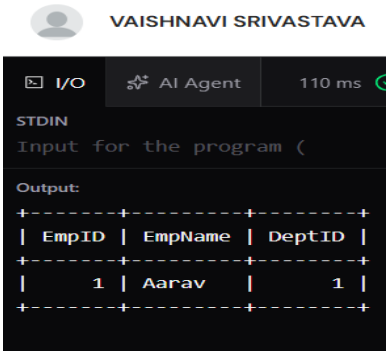
    -- Transfer employee
    UPDATE Employee
    SET DeptID = p_new_dept_id
    WHERE EmpID = p_emp_id;

    COMMIT;
END $$

DELIMITER ;
```

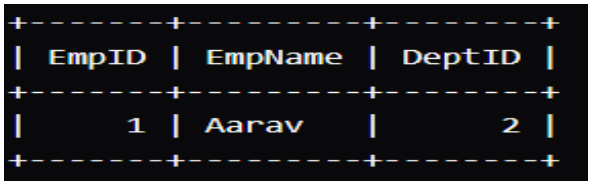
**Step 2: Test Valid Employee Transfer**

SELECT EmpID, EmpName, DeptID
FROM Employee
WHERE EmpID = 1;




**Now transfer employee 1 to Department 2:**

CALL transfer_employee(1, 2);
SELECT EmpID, EmpName, DeptID
FROM Employee
WHERE EmpID = 1;



### Step 3: Create Salary Validation Trigger

```
DROP TRIGGER IF EXISTS validate_salary_insert;

DELIMITER $$

CREATE TRIGGER validate_salary_insert
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be greater than zero';
    END IF;
END $$

DELIMITER ;
```

### Salary Update Trigger

```
DROP TRIGGER IF EXISTS validate_salary_update;

DELIMITER $$

CREATE TRIGGER validate_salary_update
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be greater than zero';
    END IF;
END $$

DELIMITER ;
```

### Step 4: Create Salary Audit Table

```
DROP TABLE IF EXISTS SalaryAudit;

CREATE TABLE SalaryAudit
(
    AuditID INT AUTO_INCREMENT PRIMARY KEY,
    EmpID INT,
    OldSalary INT,
    NewSalary INT,
    ChangedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
SELECT * FROM SalaryAudit;
```

**Step 5: Create Salary Audit Trigger**

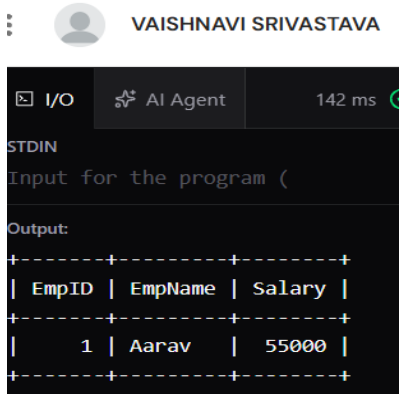
```
DROP TRIGGER IF EXISTS salary_audit;

DELIMITER $$

CREATE TRIGGER salary_audit
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.Salary <> NEW.Salary THEN
        INSERT INTO SalaryAudit
        (
            EmpID,
            OldSalary,
            NewSalary
        )
        VALUES
        (
            OLD.EmpID,
            OLD.Salary,
            NEW.Salary
        );
    END IF;
END $$
```

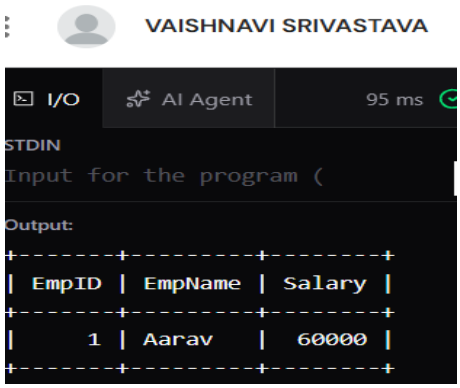
**Step 6: Test Valid Salary Update**

SELECT EmpID, EmpName, Salary
FROM Employee
WHERE EmpID = 1;

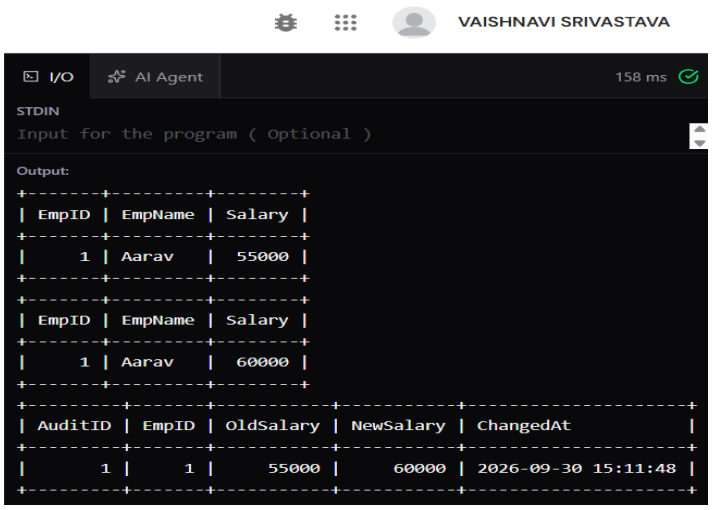



**UPDATED SALARY-**

UPDATE Employee
SET Salary = 60000
WHERE EmpID = 1;
SELECT EmpID, EmpName, Salary
FROM Employee
WHERE EmpID = 1;



**Step 7: Check Audit Log**




**Step 8: Test Invalid Salary — Zero**

UPDATE Employee
SET Salary = 0
WHERE EmpID = 1;

```
ERROR 1644 (45000) at line 242: Salary must be greater than zero
```

**Step 9: Test Invalid Salary — Negative**

UPDATE Employee
SET Salary = -5000
WHERE EmpID = 1;

```
ERROR 1644 (45000) at line 242: Salary must be greater than zero
```

**Step 10: Test Invalid Employee**

CALL transfer_employee(999, 2);

```
ERROR 1644 (45000) at line 243: Employee does not exist
```

**Step 11: Test Invalid Department**

CALL transfer_employee(1, 999);

```
ERROR 1644 (45000) at line 242: Department does not exist
```

**Step 12: Test Same Department**

CALL transfer_employee(1, 3);

```
+-------+---------+--------+
| EmpID | EmpName | DeptID |
+-------+---------+--------+
| 1 | Aarav | 3 |
+-------+---------+--------+
```